

***PROGRAMMAZIONE E
ALGORITMI
INTRODUZIONE AGLI ALGORITMI
E AI PROGRAMMI***



Cos'è la programmazione?



- Un programma descrive al computer, con estremo dettaglio, la sequenza di passi necessari per svolgere un particolare compito
- L'attività di progettare e realizzare un programma è detta programmazione.

Cos'è la programmazione?



- Usare un computer non richiede alcuna attività di programmazione, così come per guidare un'automobile non è necessario essere un meccanico
- Invece, un informatico professionista solitamente svolge una intensa attività di programmazione, anche se la programmazione non è l'unica competenza che deve avere
- La programmazione è una parte importante dell'informatica, ed è un'attività che in genere affascina gli studenti e li motiva allo studio

Cos'è un algoritmo?

- Quale tipo di problemi è possibile risolvere con un computer?
 - ▣ Dato un insieme di fotografie di paesaggi, qual è il paesaggio più rilassante?
 - ▣ Avendo depositato venti milioni in un conto bancario che produce il 5% di interessi all'anno, capitalizzati annualmente, quanti anni occorrono affinché il saldo del conto arrivi al doppio della cifra iniziale?
- Il primo problema non può essere risolto dal computer.

Perché?

Cos'è un algoritmo?



- Il primo problema non può essere risolto dal computer perché non esiste una definizione univoca di paesaggio rilassante che possa essere usata per confrontare in modo oggettivo due paesaggi diversi
- Un computer può risolvere soltanto problemi che potrebbero essere risolti anche **manualmente**: è solo molto più veloce, non si annoia, non fa errori!
- Il secondo problema è certamente risolvibile manualmente, facendo un po' di calcoli...

Cos'è un algoritmo?



- Si dice algoritmo la descrizione di un metodo di soluzione di un problema che
 - ▣ sia eseguibile
 - ▣ sia priva di ambiguità
 - ▣ giunga a una conclusione in un tempo finito
- Un computer può risolvere soltanto problemi per i quali sia noto un algoritmo

Un esempio di algoritmo

- Problema: avendo depositato venti milioni in un conto bancario che produce il 5% di interessi all'anno, capitalizzati annualmente, quanti anni occorrono affinché il saldo del conto arrivi al doppio della cifra iniziale?
- • Algoritmo:
 - ▣ 1. L'anno attuale è 0; il saldo attuale è 20M
 - ▣ 2. Ripetere i successivi passi 3 e 4 finché il saldo è minore di 40M, poi passare al punto 5
 - ▣ 3. Aggiungere 1 al valore dell'anno attuale
 - ▣ 4. Il nuovo saldo attuale è il valore del saldo attuale moltiplicato per 1.05 (cioè aggiungiamo il 5%)
 - ▣ 5. Il risultato è il valore dell'anno attuale

Un esempio di algoritmo



- Il metodo di soluzione proposto
 - ▣ è non ambiguo, perché fornisce precise istruzioni su cosa bisogna fare ad ogni passaggio e su quale deve essere il passaggio successivo
 - ▣ è eseguibile, perché ciascun passaggio può essere eseguito concretamente (se, ad esempio, il metodo di soluzione dicesse che il tasso di interesse da usare al punto 4 è variabile in dipendenza da fattori economici futuri, il metodo non sarebbe eseguibile...)
- arriva a conclusione in un tempo finito, perché ad ogni passo il saldo aumenta di almeno un milione, quindi al massimo in 20 passi arriva al termine

A cosa servono gli algoritmi?



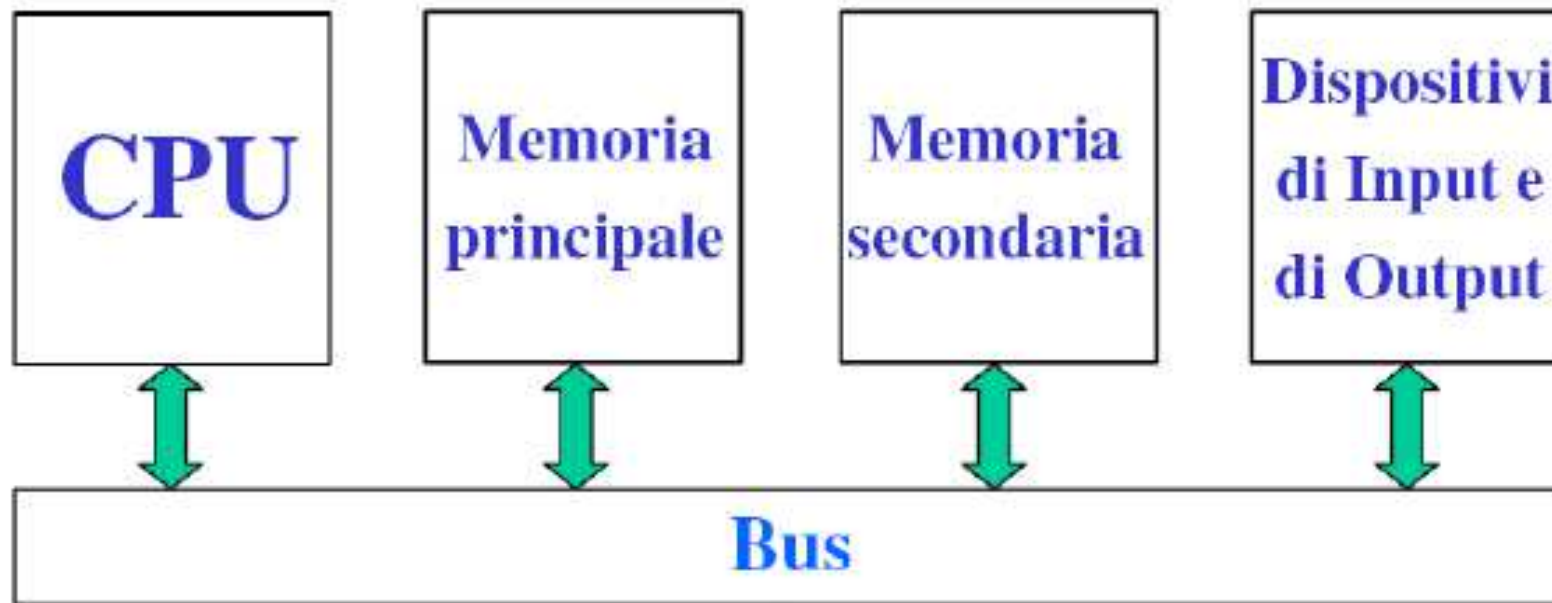
- L'identificazione di un algoritmo è un requisito indispensabile per risolvere un problema con il computer
- La scrittura di un programma per risolvere un problema con il computer consiste, in genere, nella traduzione di un algoritmo in un qualche linguaggio di programmazione
- Conclusione: prima di scrivere un programma, è necessario individuare un algoritmo!

Il modello di John von Neumann

- Nel 1946, John von Neumann elaborò un modello teorico dell'architettura di un elaboratore che è tuttora valido e molto utilizzato
- La grande maggioranza degli elaboratori odierni ha una architettura che può essere (più o meno facilmente) ricondotta al modello di von Neumann
 - ▣ le eccezioni più importanti sono alcune macchine ad elaborazione parallela
- Il modello è importante in quanto schematizza in modo omogeneo situazioni diverse
 - ▣ È presentato in una versione lievemente modificata

Il modello di John von Neumann

- L'architettura di von Neumann è composta da quattro blocchi che comunicano tra loro per mezzo di un bus, cioè un canale di scambio di informazioni.



Il bus nel modello di von Neumann



- Il bus è in realtà costituito da tre bus distinti
 - ▣ bus dei dati
 - ▣ bus degli indirizzi
 - ▣ bus dei segnali di controllo
- Sul bus dei dati viaggiano dati da e verso la CPU
- Sugli altri viaggiano segnali che provengono soltanto dalla CPU

Le istruzioni macchina



- Le istruzioni elementari eseguite da un computer (dalla sua CPU) si chiamano istruzioni macchina
- L'insieme di istruzioni macchina (instruction set) è specifico di una particolare CPU: quello di un PentiumTM è diverso da quello di uno SPARCTM

Le istruzioni macchina

- Esempio di alcune istruzioni macchina:
 - ▣ carica in un registro il valore contenuto nella posizione di memoria 40
 - ▣ carica in un altro registro il valore 100
 - ▣ se il primo valore è maggiore del secondo, prosegui con l'istruzione contenuta nella posizione di memoria 240, altrimenti con l'istruzione che segue quella attuale
- La codifica delle istruzioni macchina avviene sotto forma di configurazioni di bit conservate in memoria, che possono essere viste come numeri

Le istruzioni macchina

- In tutte le CPU, le istruzioni macchina si possono suddividere nelle seguenti categorie
 - ▣ trasferimento dati, tra i registri e la memoria principale
 - LOAD (verso un registro), STORE (verso la memoria)
 - ▣ operazioni, eseguite dalla ALU
 - aritmetiche: ADD, SUB, MUL, DIV
 - logiche: AND, OR, NOT
 - ▣ istruzioni di salto, per alterare il flusso di esecuzione sequenziale (viene modificato il Program Counter)
 - incondizionato (JUMP), salta in ogni caso
 - condizionato: salta solo se un certo valore è zero (JZ)

Le istruzioni macchina



- Per eseguire un programma in un computer è necessario scrivere all'interno della memoria primaria le configurazioni di bit corrispondenti alle istruzioni macchina del programma

- Per fare ciò è necessario conoscere tutti i codici numerici delle istruzioni macchina
 - ▣ Questa operazione lunga e noiosa (che veniva eseguita agli albori dell'informatica) è stata presto automatizzata da un programma in esecuzione sul computer stesso, detto assemblatore (assembler).

L'assemblatore



- Utilizzando l'assemblatore, il programmatore scrive il programma usando dei nomi abbreviati (codici mnemonici) per le istruzioni macchina, molto più facili da ricordare.
- L'uso di nomi abbreviati è assai più agevole e il programma assemblatore si occupa poi di tradurre il programma in configurazioni di bit.
- Tali linguaggi con codici mnemonici si dicono **linguaggi assembly** (sono diversi per ogni CPU).

L'assemblatore

- Altra caratteristica molto utile dell'assemblatore consiste nella possibilità di assegnare dei nomi agli indirizzi di memoria e ai valori numerici e di usarli nelle istruzioni.

```
iload      rate
bipush     maxRate
if_icmpgt  tooMuch
```

- In questo modo il programma è molto più leggibile, perché viene evidenziato il significato degli indirizzi di memoria e dei valori numerici.

I linguaggi assembly



- **Benefici:** rappresentarono un grosso passo avanti rispetto alla programmazione in linguaggio macchina
- **Costi:**
 - ▣ occorrono molte istruzioni per eseguire anche le operazioni più semplici
 - ▣ la sequenza di istruzioni di uno stesso programma cambia al cambiare della CPU
- **Conclusione:** è molto oneroso scrivere programmi che possano funzionare su diverse CPU (perché praticamente bisogna riscriverli completamente)

Linguaggi ad alto livello

- Negli anni '50 furono inventati i primi linguaggi di programmazione ad alto livello.
 - ▣ FORTRAN: primo “vero” linguaggio
 - BASIC, COBOL
- Anni '60 e '70: programmazione strutturata
 - ▣ Pascal (Niklaus Wirth, 1968)
 - ▣ C (Brian Kernigham e Dennis Ritchie, 1970-75)
- Anni '80 e '90, programmazione orientata agli oggetti
 - ▣ C++ (Bjarne Stroustrup, 1979)
 - ▣ Java (James Gosling e Patrick Naughton, 1991)
 - ▣ Il programmatore esprime la sequenza di operazioni da compiere, senza scendere al livello di dettaglio delle istruzioni macchina

```
if (rate > 100)
    printf("Troppo");
```

Linguaggi ad alto livello

- Un programma, detto compilatore, legge il programma in linguaggio ad alto livello e genera il corrispondente programma nel linguaggio macchina di una specifica CPU
- I linguaggi ad alto livello sono
INDIPENDENTI DALLA CPU
- Il prodotto della compilazione (codice eseguibile) invece DIPENDE dalla CPU
 - ▣ Cioè occorre compilare il programma con un diverso compilatore per ogni CPU sulla quale si vuole eseguire il programma stesso

Linguaggi ad alto livello

- Esistono molti linguaggi di programmazione ad alto livello, così come esistono molte lingue
 - ▣ L'esempio seguente è in linguaggio C

```
if (rate > 100) printf("Troppo");
```
 - ▣ questo è l'equivalente in linguaggio Pascal

```
if rate > 100 then write ('Troppo');
```
- La sintassi di un linguaggio fu scelta dai progettisti del linguaggio stesso, come compromesso fra leggibilità, facilità di compilazione e coerenza con altri linguaggi

Linguaggi ad alto livello



- I linguaggi di programmazione, creati dall'uomo, hanno una sintassi molto più rigida di quella dei linguaggi naturali
- Il compilatore segnala gli **errori di sintassi** e di **semantica** e non tenta di comprendere, come invece farebbe un utente umano

***PROGRAMMAZIONE E
ALGORITMI
DIAGRAMMI A BLOCCHI E PSEUDO
CODIFICA***



Algoritmi e programmi



- Dato un problema, la sua soluzione può essere ottenuta mediante l'uso del computer, compiendo i seguenti passi:
 1. Individuazione di un procedimento risolutivo
 2. Scomposizione del procedimento in insieme ordinato di azioni (algoritmo)
 3. Rappresentazione dei dati e dell'algoritmo attraverso un formalismo comprensibile per l'elaboratore (linguaggio di programmazione → programma) il procedimento in genere è complesso e non univoco

Operazioni fondamentali



- Le operazioni di base per la realizzazione di un algoritmo sono 4:
 1. trasferimento di informazioni: acquisizione dei dati, visualizzazione dei risultati intermedi, scrittura dei risultati finali
 2. esecuzione dei calcoli
 3. assunzione di decisioni: scelta della successiva operazione da compiere sulla base dei risultati intermedi
 4. esecuzione di iterazioni: ripetizione di sequenze determinate di operazioni

Rappresentazione degli algoritmi



- Per rappresentare un algoritmo non è possibile usare il linguaggio naturale in quanto questo può presentare ambiguità e causare false interpretazioni.
- È necessario pertanto usare linguaggi sintetici e standardizzati per consentire all'esecutore una interpretazione univoca dei passi da eseguire.

Rappresentazione degli algoritmi

- Per la descrizione degli algoritmi si usano opportune rappresentazioni grafiche denominate diagrammi di flusso, schemi a blocchi o **flowchart**.
- Tale descrizione costituisce un efficace strumento per la descrizione degli algoritmi, più valido di una esposizione di tipo discorsivo (generica e ambigua)
- Qualsiasi algoritmo può essere decomposto in poche strutture elementari.

SONO FORMALISMI CHE RAPPRESENTANO GLI ALGORITMI GRAFICAMENTE

Diagrammi di flusso

- Sono una qualsiasi combinazione di blocchi operativi e decisionali
- Regola: ogni arco (collegamento) uscente da un blocco entra in un unico, solo blocco.
- Flusso di controllo: è l'ordine di percorrenza dei blocchi individuato da un diagramma di flusso.
- Struttura di controllo: parte di un flowchart da assumere come modello elaborativo, con un ingresso e un'uscita.
- Sequenza di elaborazione: successione di blocchi operativi e decisionali prodotta dall'esecuzione di un flowchart per un determinato insieme di dati in ingresso.

Programmazione strutturata



- È una tecnica di programmazione che ha lo scopo di *semplificare la struttura* di un algoritmo attraverso l'uso di un numero limitato di strutture di controllo utilizzabili all'interno dello schema.
- Le strutture di controllo fondamentali sono:
 - 1. Sequenza**
 - 2. Selezione**
 - 3. Iterazione**ciascuna delle quali ha un solo ingresso e una sola uscita

Programmazione strutturata



- La programmazione strutturata vincola all'uso delle strutture di controllo, ma offre i seguenti vantaggi:
 - ▣ rende possibile una progettazione di tipo top-down;
 - ▣ permette la definizione di algoritmi più leggibili
 - è più facile individuare i moduli corrispondenti alle varie parti di cui si compone l'algoritmo;
 - ▣ le fasi di test, correzione e manutenzione del programma sono più semplici
 - anche se per il test completo del sistema bisogna attendere di assemblare tutti i componenti.

Progettazione top-down

- Si parte dall'**obiettivo finale** del problema da risolvere, lo si scompone in sotto problemi e si raffinano ricorsivamente le parti che lo compongono:
 - ▣ la strategia per risolvere il problema origina dall'obiettivo del problema stesso;
 - ▣ a ogni passo sono identificati dei sotto-blocchi logici correlati che vengono via via rifiniti (decomposizione, specializzazione, specificazione);
 - ▣ il processo termina quando si raggiunge il livello di dettaglio sufficiente per l'applicazione da risolvere;
 - ▣ tecnica adatta a problemi complessi di tipi diversi;
 - ▣ contrapposta alla progettazione bottom-up.

Diagrammi di flusso

- Un diagramma di flusso (flowchart) descrive le azioni da eseguire e il loro ordine di esecuzione.
- Ciascuna azione elementare corrisponde a un diverso simbolo grafico - blocco (queste convenzioni non sono universali).
- Ogni blocco ha uno o più rami in ingresso e un ramo in uscita (a esclusione dei blocchi decisionali, che ne hanno due).
- Collegando tra loro i vari blocchi attraverso i rami, si ottiene un diagramma di flusso.
- Un diagramma di flusso appare, quindi, come un insieme di blocchi di forme diverse che contengono le istruzioni da eseguire, collegati fra loro da linee orientate che specificano la sequenza in cui i blocchi devono essere eseguiti (sequenza di elaborazione).

Diagrammi di flusso

- l'insieme dei dati di ingresso e dei risultati vengono rappresentati attraverso dei nomi simbolici, detti variabili.

per esempio: a, b, c, somma, ecc.

- talvolta può essere necessario l'uso di variabili temporanee, necessarie alla risoluzione del problema: anch'esse sono rappresentate da nomi simbolici.

per esempio: delta, somma parziale, ecc.

Valori e grandezze



□ Valori:

- ▣ numerici: interi e reali (per esempio 10, oppure 23.4)
- ▣ logici: Vero o Falso
- ▣ alfanumerici o stringhe (es. “AAAA”, “Marco”)

□ Grandezze:

- ▣ Variabili: sono rappresentate da un nome simbolico a cui è assegnato un valore. Quest'ultimo può cambiare durante l'esecuzione dell'algoritmo.
- ▣ Costanti: sono quantità note a priori, il cui valore non cambia durante l'esecuzione.

Espressioni

- Sono sequenze di variabili e costanti combinate fra loro mediante operatori (per esempio, operatori aritmetici: $+$, $-$, $*$, $/$).

per esempio: $s + r * 5$

- Nella valutazione di un'espressione, si sostituisce a ogni variabile il suo valore reale e si eseguono le operazioni secondo un ordine prestabilito da regole di precedenza; si possono usare le parentesi per gestire l'ordine di esecuzione.

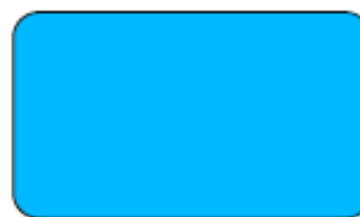
Espressioni

- A tutte le variabili che compaiono nell'espressione deve essere associato un valore prima della valutazione dell'espressione.
- Tipi di espressioni:
 - ▣ Espressioni aritmetiche: danno come risultato un valore aritmetico (per esempio, un valore intero, reale e così via...): $+$, $-$, $*$, $/$, ...
 - ▣ Espressioni relazionali e logiche: danno come risultato vero o falso ($>$, $<$, $=$, $\geq$, $\leq$, $<>$)

Blocchi



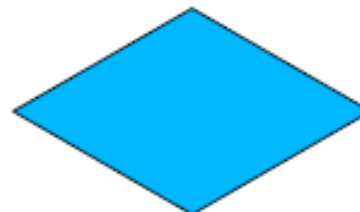
ingresso/uscita



inizio/fine



elaborazione/calcolo



decisione



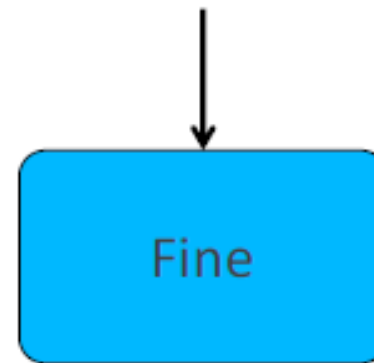
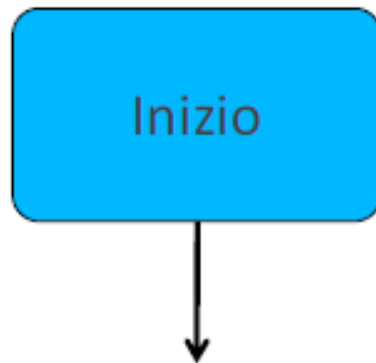
elaborazione predefinita



connessioni

Inizio/fine

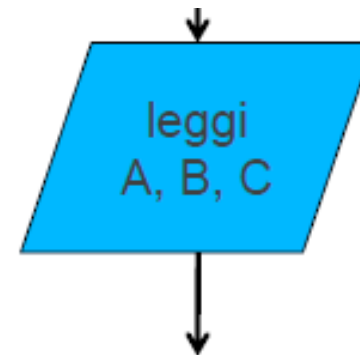
- Inizio e fine di un algoritmo
 - ▣ L'inizio è il blocco da cui deve iniziare l'esecuzione (uno solo)
 - ▣ Il blocco fine fa terminare l'esecuzione dell'algoritmo (almeno uno in un algoritmo)



Ingresso/lettura

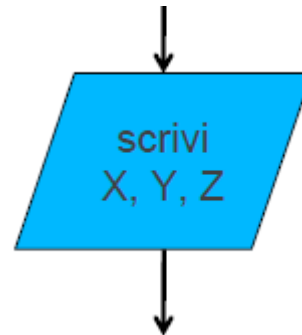
□ Esecuzione dell'istruzione:

- Si ricevono dall'unità di ingresso (per esempio, la tastiera) tanti valori quante sono le variabili specificate all'interno del blocco, e si assegnano nello stesso ordine alle variabili
- A, B, C sono nomi di variabili. Per esempio, “Leggi i tre valori forniti in ingresso, e assegnali rispettivamente alle variabili A, B, e C”



Uscita/stampa

- si calcolano i valori delle espressioni e si trasmettono all'unità di uscita (es. il video)
 - ▣ X, Y, Z possono essere variabili
 - ▣ se X, Y, Z sono espressioni → “calcola i valori delle espressioni X, Y e Z, e trasmettiti in uscita”



- N.B: i valori di X, Y, Z non sono alterati all'esecuzione del blocco

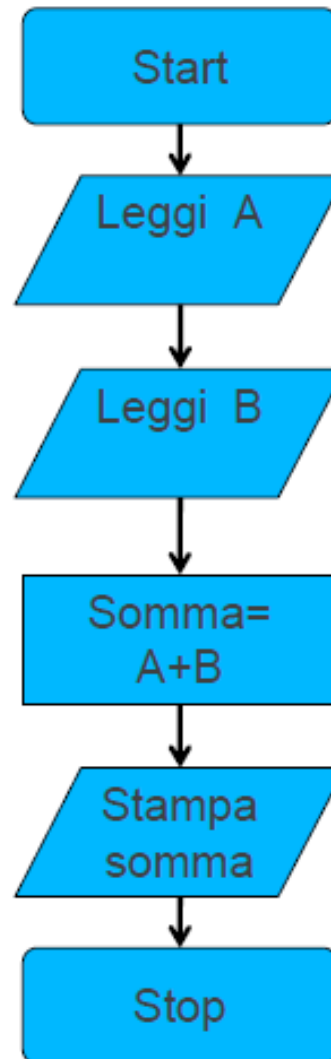
Assegnazione

- Si calcola il valore dell'espressione a destra del simbolo “=” e lo si assegna alla variabile indicata a sinistra del simbolo “=”, con eventuale perdita del valore precedente della variabile.

$$V = E$$

- Dove
 - ▣ V è il nome di una variabile
 - ▣ E è un'espressione
- Si interpreta come: “calcola il valore dell'espressione E e assegnalo alla variabile V”

Esempio: somma di due numeri



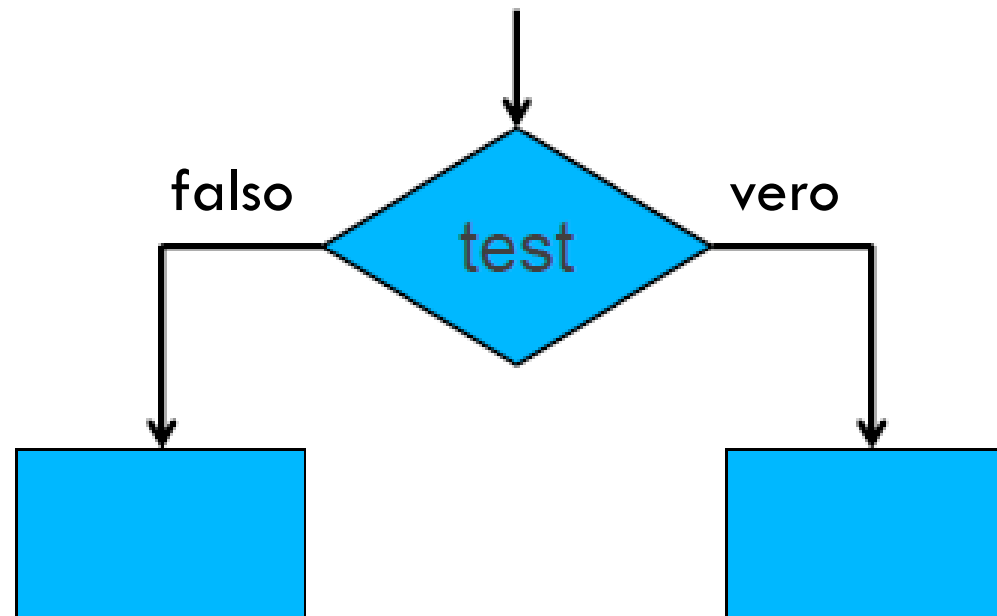
Strutture di controllo



- Mediante i blocchi fondamentali, è possibile costruire delle strutture tipicamente utilizzate per il controllo del flusso di esecuzione dell'algoritmo.
- Le strutture di controllo fondamentali sono:
 - ▣ Selezione
 - ▣ Iterazione

Strutture di controllo: selezione

- Esprime la scelta tra due possibili azioni, o sequenze di azioni, mutuamente esclusive:

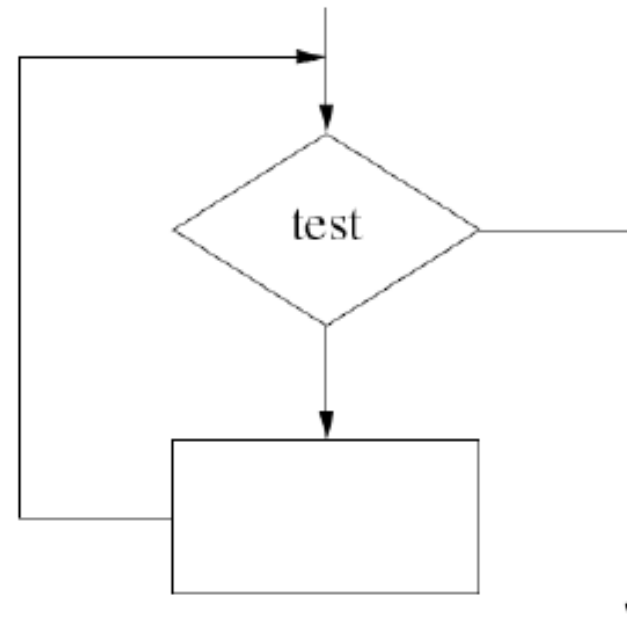
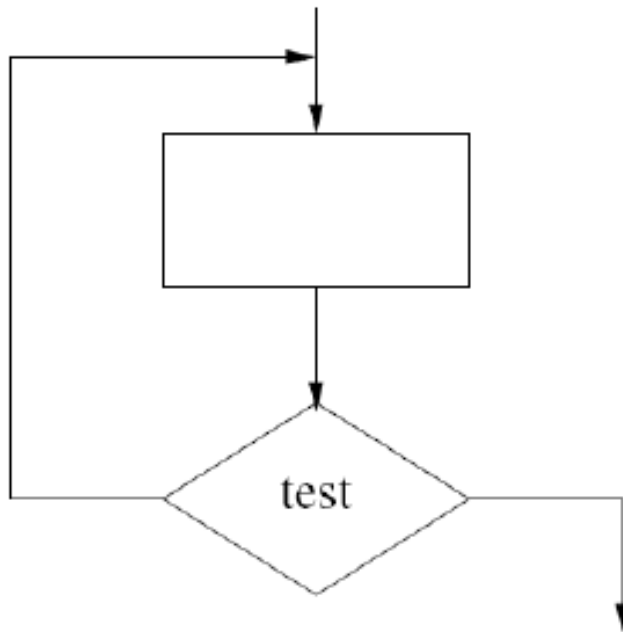


Strutture di controllo: iterazione

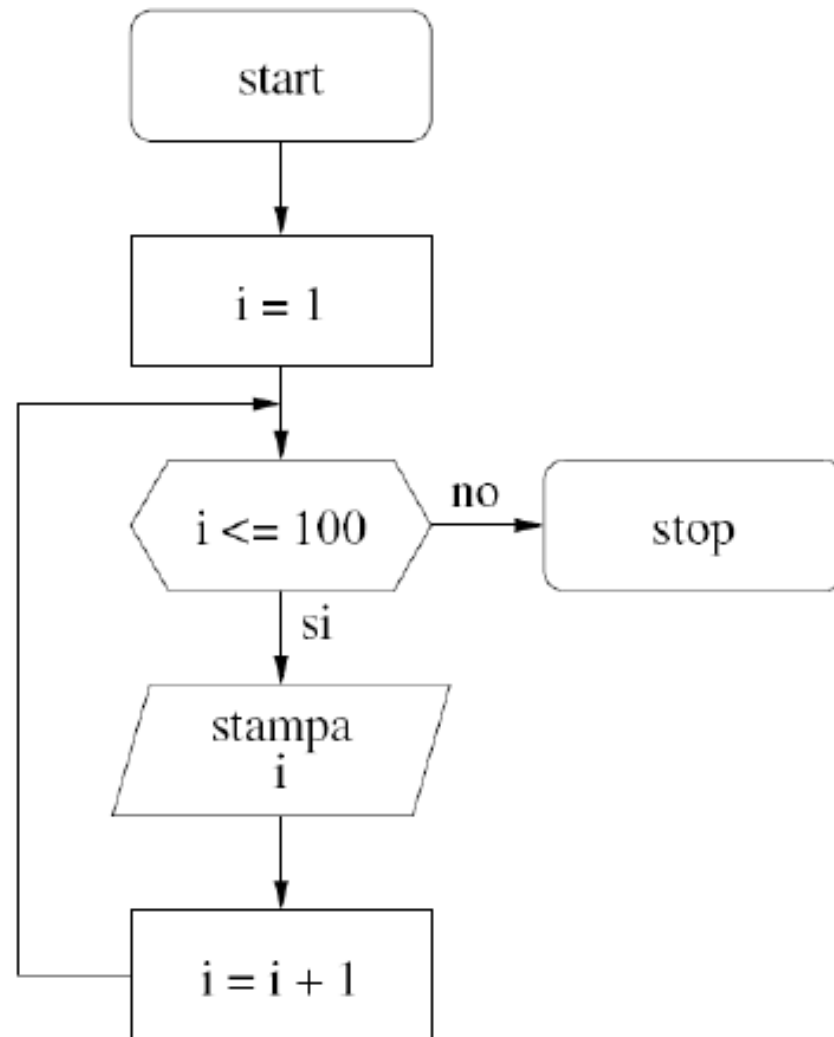
- la ripetizione di una sequenza di istruzioni
- In generale, è costituita da:
 - ▣ inizializzazione: assegnazione dei valori iniziali alle variabili
 - ▣ caratteristiche del ciclo (viene eseguita una sola volta)
 - ▣ corpo: esecuzione delle istruzioni fondamentali del ciclo che devono essere eseguite in modo ripetitivo
 - ▣ modifica: modifica dei valori delle variabili che controllano l'esecuzione del ciclo (eseguito a ogni iterazione)
 - ▣ controllo: determina, in base al valore delle variabili che controllano l'esecuzione del ciclo se il ciclo deve essere ripetuto o meno.

Strutture di controllo: iterazione

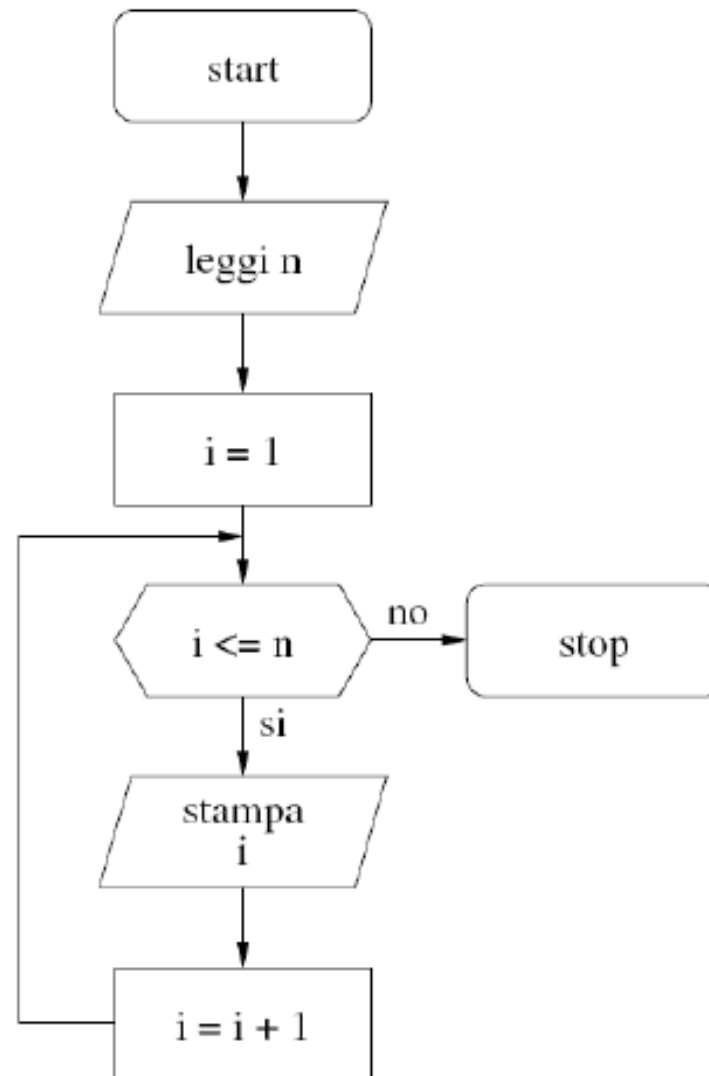
- sono possibili due configurazioni:
 - ▣ Lavora e poi controlla
 - ▣ Controlla e poi lavora



Esempio: stampa dei primi 100 numeri



Esempio: stampa dei primi n numeri



Controllo del flusso

- permettono di decidere se e come eseguire blocchi di istruzioni vincolando a opportuni meccanismi decisionali

Istruzione	descrizione
<code>if</code>	esegue o meno un blocco di codice a seconda del valore restituito da una espressione booleana
<code>if-else</code>	permette di selezionare tra due blocchi di codice quello da eseguire secondo il valore restituito da una espressione Booleana
<code>switch</code>	utile in tutti quei casi in cui sia necessario decidere tra opzioni multiple prese in base al controllo di una sola Variabile
<code>for</code>	esegue ripetutamente un blocco di codice
<code>while</code>	esegue ripetutamente un blocco di codice controllando inizialmente il valore di una espressione booleana (test all'inizio del ciclo)
<code>do-while</code>	esegue ripetutamente un blocco di codice controllando il valore di una espressione booleana dopo una esecuzione iniziale



Esempi

Uso dei loop

Somma tra due numeri positivi

Pseudocodice (ciclo for):

```
leggi X e Y
INTERO SOMMA
INTERO I
SOMMA = X
FOR (I = 1; I <= Y; I=I+1) {
    SOMMA = SOMMA + 1
}
stampa SOMMA
```

La somma di due numeri positivi può essere espressa attraverso il concetto dell'incremento unitario: $n+m$ equivale a incrementare m , di un'unità, per n volte, oppure incrementare n per m volte.

L'algoritmo risolutivo è eccessivamente lungo, ma utile per apprendere il funzionamento dei cicli.

Somma tra due numeri positivi (2)

Pseudocodice (ciclo while):

```
leggi X e Y
```

```
INTERO SOMMA
```

```
INTERO I
```

```
SOMMA = X
```

```
I = 1
```

```
WHILE (I <= Y) {
```

```
    SOMMA = SOMMA + 1
```

```
    I=I+1
```

```
}
```

```
stampa SOMMA
```

Moltiplicazione di due numeri positivi attraverso la somma (1)

Pseudocodice (ciclo for):

leggi X e Y

INTERO PRD

INTERO I

PRD = 0

FOR (I = 1; I <= Y; I=I+1) {

 PRD = PRD + X

}

stampa PRD

La moltiplicazione di due numeri positivi può essere espressa attraverso il concetto della somma.

Il prodotto di $n \times m$ equivale a sommare m volte n , oppure n volte m .

L'algoritmo risolutivo è eccessivamente lungo, ma utile per apprendere il funzionamento dei cicli.

Moltiplicazione di due numeri positivi attraverso la somma (2)

Pseudocodice (ciclo while):

leggi X e Y

INTERO PRD

INTERO I

PRD = 0

I = 1

WHILE (I <= Y) {

 PRD = PRD + X

 I = I + 1

}

stampa PRD

Elevamento a potenza (1)

Pseudocodice (ciclo for):

```
leggi X e Y
INTERO POW
INTERO I
POW = 1
FOR (I = 1; I <= Y; I= I+1) {
    POW = POW * X
}
stampa POW
```

L'elevamento a potenza, utilizzando numeri positivi, può essere espresso attraverso il concetto della moltiplicazione. n elevato a m equivale a moltiplicare n per se stesso, m volte

Elevamento a potenza (2)

Pseudocodice (ciclo while):

```
leggi X e Y
INTERO POW
INTERO I
POW = 1
WHILE (I <= Y) {
    POW = POW * X
    I = I + 1
}
stampa POW
```

Radice quadrata

Pseudocodice (ciclo while):

```
Leggi X
INTERO RDC
INTERO T
RDC = 0
T = 0
WHILE (TRUE) {
    T = RDC * RDC
    IF (T > X) {
        // È stato superato il valore massimo.
        RDC = RDC - 1
        stampa RDC
    }
    RDC = RDC + 1
}
```

Il calcolo della parte intera della radice quadrata di un numero si può fare per tentativi, partendo da 1, eseguendo il quadrato fino a quando il risultato è minore o uguale al valore di partenza di cui si calcola la radice.

Fattoriale

Pseudocodice (ciclo while):

```
leggi X
INTERO I
I = X - 1
WHILE (I > 0)
    X = X * I
    I = I - 1
}
stampa X
```

Il fattoriale è un valore che si calcola a partire da un numero positivo. Può essere espresso come il prodotto di n per il fattoriale di $n-1$, quando n è maggiore di 1, e solo 1 quando n è uguale a 1.

In pratica

$$n! = n * (n - 1) * (n - 2) \dots * 1$$

Massimo comune divisore

Pseudocodice (ciclo while):

```
Leggi X e Y
WHILE (X != Y) {
    IF (X > Y) {
        X = X - Y
    ELSE
        Y = Y - X
    }
}
stampa X
```

Il massimo comune divisore tra due numeri può essere ottenuto sottraendo a quello maggiore il valore di quello minore, fino a quando i due valori sono uguali.

Quel valore è il massimo comune divisore.

Indovina il numero

Pseudocodice (ciclo while):

```
ESTRAI RANDOMICAMENTE N
INTERO CONT = 0
INTERO X
DO {
    SCRIVI "INSERISCI UN NUMERO"
    LEGGI X
    IF (X<N) {
        SCRIVI "NUMERO TROPPO PICCOLO"
    }
    IF (X>N) {
        SCRIVI "NUMERO TROPPO GRANDE"
    }
    CONT=CONT+1
} WHILE (X != N)
STAMPA "INDOVINATO IN " + CONT + " TENTATIVI"
```

Array

- Oltre alle singole variabili, in programmazione è possibile utilizzare gli array
- Un array è una variabile strutturata, cioè una variabile composta da diverse caselle

10	5	64	2	54	12	9	1	34	7
0	1	2	3	4	5	6	7	8	9

- Le sue caratteristiche sono:
 - ▣ Ha un unico nome
 - ▣ Ogni casella è individuabile attraverso un indice
 - ▣ Ha una lunghezza

Accesso e operazioni sugli array

- L'accesso agli elementi si effettua attraverso l'indice.

`legge array[2]` → legge l'elemento di posto 2

`array[2] = 4` → scrive 4 nel posto 2 dell'array

Le più comuni operazioni sugli array sono:

- Letture
 - Scritture
 - Ricerche
 - Ordinamenti
- Si effettuano indicando il nome dell'array e l'indice
- Si effettuano con opportune funzioni che implementano algoritmi specifici

Ricerca sequenziale su un array

- La ricerca di un elemento all'interno di un array disordinato può avvenire solo in modo sequenziale, cioè controllando uno per uno tutti gli elementi, fino a quando si trova la corrispondenza cercata.
- Variabili
 - ▣ **LISTA:** È l'array su cui effettuare la ricerca.
 - ▣ **X:** È il valore cercato all'interno dell'array.
 - ▣ **A:** È l'indice inferiore dell'intervallo di array su cui si vuole effettuare la ricerca.
 - ▣ **Z:** È l'indice superiore dell'intervallo di array su cui si vuole effettuare la ricerca

Ricerca sequenziale su un array

Pseudocodice (ciclo for):

```
LEGGI ARRAY, X, A, Z
INTERO I
FOR (I := A; I <= Z; I=I+1) {
    IF (X == LISTA[I]) {
        STAMPA I // corrispondenza trovata.
    }
}
// corrispondenza non trovata.
STAMPA -1
}
```

Ordinamenti



- Esistono numerosi algoritmi di ordinamento di un array:
 - ▣ Selection sort
 - ▣ Bubble sort
 - ▣ Quick sort
- Il Bubble sort, in particolare, si basa sull'idea di far “salire” gli elementi più piccoli (“più leggeri”) verso l'inizio del vettore (“verso l'alto”), ` scambiandoli con quelli adiacenti

Ordinamento con Bubble sort

```
Leggi LISTA, A, Z // A indice inizio, Z indice fine
INTERO J
INTERO K
// Scandisce l'array attraverso l'indice J in modo da
// collocare ogni volta il valore corretto all'inizio
// dell'array stesso.
FOR (J = A; J < Z; J=J+1) {
    // Scandisce l'array attraverso l'indice K scambiando i
    // valori quando sono inferiori a quello di riferimento.
    FOR (K = J+1; K <= Z; K=K+1) {
        IF (LISTA[K] < LISTA[J]) {
            // I valori vengono scambiati.
            LISTA[K]  $\leftrightarrow$  LISTA[J]
        }
    }
}
```